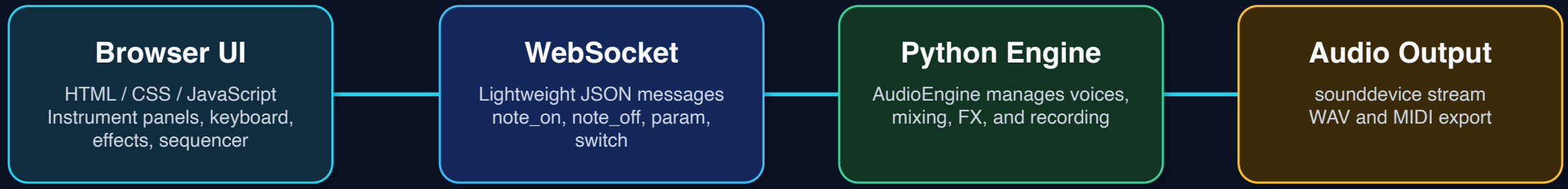
ACTAM PRO

Project Framework

A web-based real-time audio workstation where the browser controls musical events and Python renders the sound.

WEB CONTROLS | BACKEND SOUNDS | REAL-TIME DSP



Saeid Soleimani, 11024150, saeid.soleimani@mail.polimi.it
Hanxiang Gao, 11088786, hanxiang.gao@mail.polimi.it

ARCHITECTURE

Project Layers and Signal Flow

LAYERED DESIGN

UI — index.html + style.css
Visible DAW-style interface

Interaction — assets/js/app.js
Transforms actions into music events

Communication — WebSocket + main.py
Sends and routes JSON messages

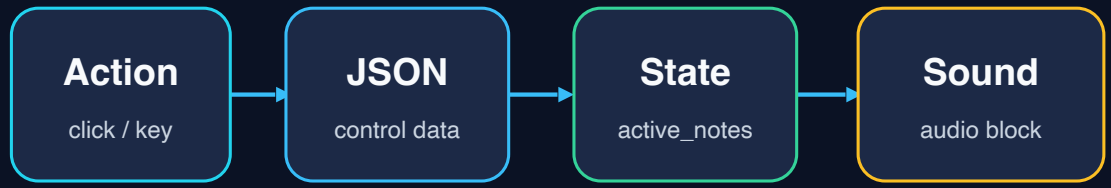
Audio Engine — server/engine.py
Manages active notes, parameters, mixing

Synthesis / FX — instruments.py + fx.py
Generates waveforms and processes master effects

Output / Export — sounddevice + recordings
Playback, WAV recording, MIDI event export

Runtime control path

1. User action in browser
2. app.js creates event object
3. AudioClient sends JSON
4. main.py routes message
5. engine.py updates state
6. audio_callback outputs block



Frontend Module

Role: interface + event organization

The frontend decides what the user wants to play, then sends structured control messages to the backend.

`AudioClient.send()` `ACTAMServer.ws_handler()`

index.html

Defines the DAW layout: instrument panels, virtual keyboard, drum machine, effects area, recording controls, and arrangement section.

style.css

Builds the visual identity: responsive layout, panels, buttons, grids, and modern workstation look.

app.js

Captures inputs, manages sequencer timing, smart chords, arrangement events, and WebSocket communication.

app.js/AudioClient

Connects to ws://localhost:8765 and sends JSON object

Frontend input sources

- Keyboard or mouse playing
- Instrument and effects controls
- Drum pattern grid
- Arrangement playback and capture

```
noteOn(baseMidi) {
    if (!this.isPlayable) return;

    // --- INTERCEPT: KBD CHORD MODE ---
    if (this.keyboardChordMode && this.activeVst !== 'drums') {
        this.playChordFromRoot(baseMidi, true);
        return;
    }

    let actualMidi = this.activeVst === 'drums' ? baseMidi : baseMidi + (this.octaveShift * 12);
    if (this.activeSet.has(actualMidi)) return;

    this.activeSet.add(actualMidi);
    this.playedNotes.set(baseMidi, actualMidi);
    if (this.keyElems[baseMidi]) this.keyElems[baseMidi].classList.add('active');

    const freq = this.midiToFreq(actualMidi);
    this.client.send({ type: 'note_on', id: actualMidi, freq: freq, vst: this.activeVst });
    if (this.arrangement) this.arrangement.recordNoteOn(this.activeVst, actualMidi, 1);

    if (this.activeVst === 'guitar') this.updateFretboard(actualMidi, true);
    if (this.activeVst === 'strings') this.updateViolinStrings(actualMidi, true);
    this.updateDisplay();
}
```

`note_on`

`note_off`

`chord_on/off`

`switch`

`param`

`start/stop_recording`

```
{
  type: "note_on",
  id: 60,
  freq: 261.63,
  vst: “piano”,
  velocity: 1.0
}
```

Smart Chords

Frontend music-theory layer. It builds scale notes and chord structures, then sends multiple notes as one musical event.

```
initChordPads() {
    this.chordTypes = {
        'Maj': [0, 4, 7], 'Min': [0, 3, 7], 'Dim': [0, 3, 6],
        'Aug': [0, 4, 8], 'Sus4': [0, 5, 7], 'Sus2': [0, 2, 7]
    };
    this.noteNames = ['C','C#','D','D#','E','F','F#','G','G#','A','A#','B'];

    // Diatonic Scale Formulas
    this.scaleFormulas = {
        'Major': [
            { int: 0, type: 'Maj', num: 'I' }, { int: 2, type: 'Min', num: 'ii' },
            { int: 4, type: 'Min', num: 'iii' }, { int: 5, type: 'Maj', num: 'IV' },
            { int: 7, type: 'Maj', num: 'V' }, { int: 9, type: 'Min', num: 'vi' },
            { int: 11, type: 'Dim', num: 'vii°' }
        ],
```

```
this.padConfigs = Array(7).fill().map(() => ({ rootIdx: 0, type: 'Maj', octave: 3, numeral: '' }));

// Init Global Selectors
const globRoot = document.getElementById('globalKeyRoot');
const globScale = document.getElementById('globalKeyScale');
if (globRoot && globScale) {
    this.noteNames.forEach((n, idx) => {
        let opt = document.createElement('option');
        opt.value = idx; opt.textContent = n;
        globRoot.appendChild(opt);
    });
    globRoot.addEventListener('change', () => this.applyGlobalKey());
    globScale.addEventListener('change', () => this.applyGlobalKey());
}

this.renderChordPads();
this.applyGlobalKey(); // Auto-calculate C Major on startup
```

Sequencer + Drum Machine

BPM-based 16th-step timing, swing, humanized velocity/delay, and three-state drum cells: off, normal, accent.

```
getTimeParts() {
    const [beats, unit] = this.timeSignature.value.split('/').map(Number);
    return { beats: beats || 4, unit: unit || 4 };
}
```

```
getStepMs() {
    const bpm = parseInt(this.bpmInput.value) || 120;
    return ((60 / bpm) * 1000) / 4;
}
```

```
class Sequencer {
    constructor(client) {
        this.client = client;
        this.audioCtx = new (window.AudioContext || window.webkitAudioContext)();
        this.seqInterval = null;
        this.currentStep = 0;
        this.playingStep = 0;
        this.lastStepTime = 0;
        this.isPlaying = false;
        this.isRecording = false;
        this.clickEnabled = true;
        this.drumMachineEnabled = true;
        this.drumLoopEnabled = true;
        this.drumPresetStoragePrefix = 'actamDrumPreset:';
        this.legacyCustomDrumPresetKey = 'actamCustomDrumPreset';
        this.activeDrumPreset = 'empty';
        this.hasUnsavedDrumChanges = false;
        this.activePatternSlot = 'A';
        this.patternSlots = { A: null, B: null };
        this.swingAmount = 0;
        this.humanizeEnabled = false;
        this.fillStepsRemaining = 0;
        this.fillStartStep = 0;
        this.onPatternChange = null;
        this.drumLanes = [
            { id: 'crash', label: 'CRASH', note: 49, key: 'R' },
            { id: 'ride', label: 'RIDE', note: 51, key: 'Y' },
            { id: 'openhat', label: 'OPEN HAT', note: 46, key: 'C' },
            { id: 'hihat', label: 'CLOSED HAT', note: 42, key: 'X' },
            { id: 'hitom', label: 'HI TOM', note: 48, key: 'F' },
            { id: 'midtom', label: 'MID TOM', note: 45, key: 'G' },
            { id: 'floortom', label: 'FLOOR TOM', note: 41, key: 'V' },
            { id: 'snare', label: 'SNARE', note: 38, key: 'S' },
            { id: 'kick', label: 'KICK', note: 36, key: 'SPACE' }
        ];
```

```
cell.addEventListener('click', () => {
    this.drumPattern[lane.id][step] = (this.drumPattern[lane.id][step] + 1) % 3;
    this.markDrumPatternEdited();
    this.renderDrumMachine();
    this.notifyDrumPatternChanged();
});
```

scheduleNextStep()

```
    if (this.clickEnabled && isBeatStep) this.playClick(this.currentStep === 0);
    if (this.drumMachineEnabled) this.playDrumStep(this.currentStep);
    if (this.arrangement) this.arrangement.handleTick();
    if (this.fillStepsRemaining > 0) {
        this.fillStepsRemaining -= 1;
        if (this.fillStepsRemaining === 0) this.updateFillButton();
    }
    this.updateDrumPlayhead();

    const nextStep = this.currentStep + 1;
    const stepDelay = this.getSwingStepMs(this.currentStep);
    if (nextStep >= totalSteps && !this.drumLoopEnabled) {
        this.seqInterval = setTimeout(() => this.togglePlay(), stepDelay);
        return;
    }

    this.currentStep = nextStep % totalSteps;
    this.seqInterval = setTimeout(() => this.scheduleNextStep(), stepDelay);
}
```

Sequencer + Drum Machine

BPM-based 16th-step timing, swing, humanized velocity/delay, and three-state drum cells: off, normal, accent.

```
triggerDrum(midiNote, velocity = 1) {
    if (!this.client) return;
    this.client.send({ type: 'note_on', id: midiNote, freq: 0, vst: 'drums', velocity });
    const pad = document.querySelector(`.drum-pad[data-note="${midiNote}"]`);
    if (pad) {
        pad.classList.add('active');
        setTimeout(() => pad.classList.remove('active'), 80);
```

```
playDrumStep(step) {
    this.drumLanes.forEach(lane => {
        const state = this.drumPattern[lane.id] ? this.drumPattern[lane.id][step] : 0;
        if (state) this.playDrumLaneAtStep(lane, step, state);
    });
    this.playFillStep(step);
}

playDrumLaneAtStep(lane, step, state) {
    const baseVelocity = state === 2 ? 1 : 0.68;
    const velocity = this.getHumanizedVelocity(baseVelocity);
    const delay = this.getHumanizedDelay();
    const trigger = () => {
        this.triggerDrum(lane.note, velocity);

        const cell = this.drumMachine ? this.drumMachine.querySelector(`.dm-cell[data-lane="${lane.id}"][data-step="${step}"]`)
        if (cell) {
            cell.classList.add('triggered');
            setTimeout(() => cell.classList.remove('triggered'), 95);
        }
    };
```

FRONTEND MODULE

Arrangement Draft

8-bar MIDI-like workspace storing track, MIDI note, start time, duration, velocity.

```
getBarSteps() {
    return this.sequencer.getTotalSteps();
}

getTotalSteps() {
    return this.getBarSteps() * this.bars;
}

getStepMs() {
    return this.sequencer.getStepMs();
}

getLoopMs() {
    return this.getTotalSteps() * this.getStepMs();
}

stepToMs(step) {
    return step * this.getStepMs();
}

msToStep(ms) {
    return Math.max(0, Math.min(this.ge
}
```

```
class Arrangement {
    constructor(client, sequencer) {
        this.client = client;
        this.sequencer = sequencer;
        this.bars = 8;
        this.tracks = ['piano', 'strings', 'guitar', 'drums'];
        this.trackColors = {
            piano: '#33ff99',
            strings: '#ffcc66',
            guitar: '#55ccff',
            drums: '#ff6677'
        };
        this.notes = this.tracks.reduce((acc, track) => ({ ...acc, [track]: [] }), {});
        this.activeTakes = new Map();
        this.recording = false;
        this.playheadStep = 0;
        this.lastTickStep = 0;
        this.lastTickAt = performance.now();
        this.playingNotes = [];
        this.editor = { track: null, bar: 0, selected: null };
        this.storageKey = 'actamArrangementDraft';
        this.initDOM();
    }
```

```
this.notes = {
  piano: [],
  strings: [],
  guitar: [],
  drums: []
}
```

```
handleTick() {
    const step = this.getCurrentStep();
    this.lastTickStep = step;
    this.lastTickAt = performance.now();
    this.startNotesInWindow(this.stepToMs(step), this.stepToMs(step + 1), step);
    this.renderPlayhead();
    this.playheadStep = (this.playheadStep + 1) % this.getTotalSteps();
    this.updateStatus();
}
```

startNotesInWindow()

```
const wrapped = endMs >= loopMs;
const windowEnd = endMs % loopMs;
this.tracks.forEach((track, trackIndex) => {
    this.notes[track].forEach((note, index) => {
        const inWindow = wrapped
            ? note.startMs >= startMs || note.startMs < windowEnd
            : note.startMs >= startMs && note.startMs < endMs;
        if (!inWindow) return;

        const offset = wrapped && note.startMs < windowEnd
            ? (loopMs - startMs) + note.startMs
            : Math.max(0, note.startMs - startMs);
        setTimeout(() => this.playNote(track, trackIndex, note, step, index), offset);
    });
```

FRONTEND MODULE

Arrangement Draft

8-bar MIDI-like workspace storing track, MIDI note, start time, duration, velocity.

```
addNote(track, midi, startMs, durationMs, velocity = 1, shouldRender = true) {
    const startStep = this.msToStep(startMs);
    const durationSteps = Math.max(1, Math.round(durationMs / this.getStepMs()));
    const note = {
        id: `${Date.now()}_${Math.random().toString(16).slice(2)}`,
        midi,
        startMs,
        durationMs,
        startStep,
        durationSteps,
        velocity
    };
    this.notes[track].push(note);
    this.sortTrack(track);
    if (shouldRender) this.render();
}
```

```
playNote(track, trackIndex, note, step, index) {
    if (track === 'drums') {
        this.client.send({ type: 'note_on', id: note.midi, freq: 0, vst: 'drums', velocity: note.velocity || 1 });
        return;
    }

    const playId = 90000 + trackIndex * 100000 + step * 100 + index;
    this.client.send({ type: 'note_on', id: playId, freq: this.midiToFreq(note.midi), vst: track, velocity: note.velocity || 1 });
    const timer = setTimeout(() => {
        this.client.send({ type: 'note_off', id: playId });
        this.playingNotes = this.playingNotes.filter(item => item.playId !== playId);
    }, Math.max(35, note.durationMs || this.getStepMs()));
    this.playingNotes.push({ ...note, playId, timer });
}
```

```
recordNoteOn(track, midi, velocity = 1) {
    if (!this.recording || !this.tracks.includes(track) || track === 'drums') return;
    const key = `${track}:${midi}`;
    if (this.activeTakes.has(key)) return;
    this.activeTakes.set(key, {
        track,
        midi,
        velocity,
        startMs: this.getCurrentMs()
    });
}

recordNoteOff(track, midi) {
    if (!this.recording || !this.tracks.includes(track) || track === 'drums') return;
    const key = `${track}:${midi}`;
    const take = this.activeTakes.get(key);
    if (!take) return;

    this.activeTakes.delete(key);
    const endMs = this.getCurrentMs();
    let durationMs = (endMs - take.startMs + this.getLoopMs()) % this.getLoopMs();
    if (durationMs < 35) durationMs = 120;
    this.addNote(take.track, take.midi, take.startMs, durationMs, take.velocity);
}
```

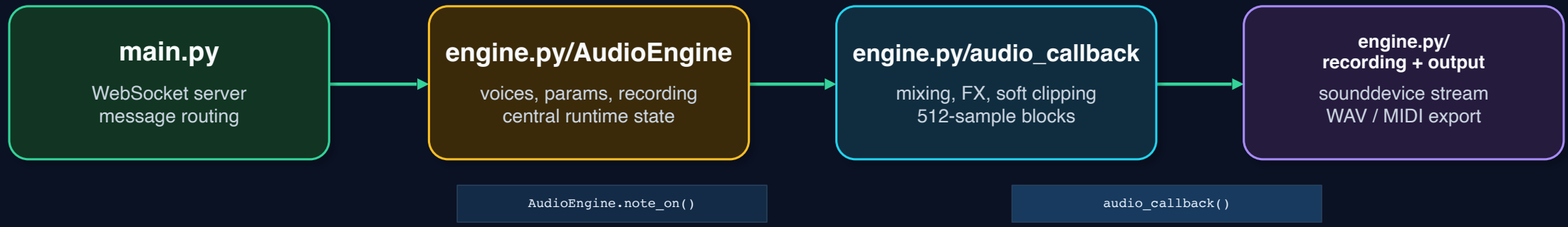
```
captureDrumPattern(sequencer, targetBar = this.getCurrentBarIndex()) {
    if (!sequencer) return;
    targetBar = Math.max(0, Math.min(this.bars - 1, targetBar));
    sequencer.ensureDrumPatternLength();
    const barMs = this.getLoopMs() / this.bars;
    const barStartMs = targetBar * barMs;
    const barEndMs = barStartMs + barMs;
    const barSteps = this.getBarSteps();

    this.notes.drums = this.notes.drums.filter(note => note.startMs < barStartMs || note.startMs >= barEndMs);
    sequencer.drumLanes.forEach(lane => {
        const pattern = sequencer.drumPattern[lane.id] || [];
        pattern.forEach((state, step) => {
            if (!state) return;
            this.addNote('drums', lane.note, this.stepToMs(targetBar * barSteps + step), this.getStepMs(), state ===
        });
    });
    this.render();
}
```

Backend Module

Role: communication router + real-time audio engine

The backend receives JSON messages, updates engine state, generates audio blocks, applies effects, and records output.



active_notes

- Dictionary of currently sounding voices
- Stores waveform data, playback position, release state, instrument type, base speed, and gain

params

Shared parameter set controlled by frontend: expression, tune, pitch bend, tremolo, delay, reverb and drum setting

recording buffers

- WAV buffers store rendered audio data
- MIDI events store midi event

STATE UPDATE

```
note_on → active_notes[id] = {data, pos, on, rel_pos, vst, midi_note, base_speed, gain}
```

AUDIO BLOCK CHAIN

```
audio_callback(): read active voices -> interpolate -> mix -> apply FX -> tanh soft clipping -> outdata
```

BACKEND MODULE

Instrument Synthesis

Piano

Additive synthesis:
Multiple partials are summed with exponential decay and slight inharmonicity for a synthetic piano tone

```
partials = [
    (1, 1.000, 0.35), (2, 0.720, 0.88), (3, 0.510, 1.75),
    (4, 0.350*brt, 3.10), (5, 0.240*brt, 4.90), (6, 0.160*brt, 7.20),
]

sig = np.zeros(n_samp, dtype=np.float32)
for h, amp, d0 in partials:
    fh = f0 * h * float(np.sqrt(1.0 + B * h * h))
    sig += amp * np.sin(2 * np.pi * fh * t) * np.exp(-d0 * (0.75 + 0.50 * norm) * t)
```

```
    f0 = self.midi2freq(midi_note)
    N = max(2, int(round(self.sr / f0)))
    exc = np.random.default_rng(int(midi_note)).standard_normal(N).astype(np.float64)
    a_coef = np.zeros(N + 2, dtype=np.float64)
    a_coef[0], a_coef[N], a_coef[N+1] = 1.0, -0.495, -0.495
    x_buf = np.zeros(n_out, dtype=np.float64)
    x_buf[:N] = exc
    output = lfilter([1.0], a_coef, x_buf).astype(np.float32)

    try:
        sos_lp = butter(2, 7000 / (self.sr/2), btype='low', output='sos')
        warm = sosfilt(sos_lp, output.astype(np.float64)).astype(np.float32)
        output = 0.72 * output + 0.28 * warm
    except Exception: pass
```

Strings

Supersaw-style ensemble pad:
Detuned oscillators, vibrato, slow attack, and filtering

```
f0 = self.midi2freq(midi_note)
sig = np.zeros(n_samp, dtype=np.float32)

vibrato = 0.003 * np.sin(2 * np.pi * 5.0 * t)

for d in [-0.15, 0.0, 0.15]:
    freq = f0 + (f0 * d * 0.01)
    phase = np.cumsum((freq * (1.0 + vibrato)) / self.sr)
    sig += (2.0 * (phase % 1.0) - 1.0) * 0.3

attack_len = int(0.25 * self.sr)
if attack_len > 0 and len(sig) > attack_len:
    sig[:attack_len] *= np.linspace(0.0, 1.0, attack_len, dtype=np.float32)

try:
    sos = butter(2, 4000 / (self.sr/2), btype='low', output='sos')
    sig = sosfilt(sos, sig)
except Exception: pass
```

Guitar

Plucked-string physical modeling:
Noise excitation circulates through a feedback delay with low-pass energy loss

Drums

Procedural drums. Envelopes, sine waves, noise, and filtering

```
if midi_note == 36: # Kick
    freqs = 150.0 * np.exp(-30.0 * t) + 40.0
    sig = np.sin(2 * np.pi * np.cumsum(freqs / self.sr)) * np.exp(-4.0 * t)
elif midi_note == 38: # Snare
    tone = np.sin(2 * np.pi * 180 * t) * np.exp(-12.0 * t)
    noise = np.random.normal(0, 1, len(t)) * np.exp(-25.0 * t)
    try:
        sos = butter(2, 1000 / (self.sr/2), btype='high', output='sos')
        noise = sosfilt(sos, noise)
    except Exception: pass
    sig = tone + (noise * 0.8)
elif midi_note in [42, 46]: # Hi-Hat
    decay = 35.0 if midi_note == 42 else 5.0
    noise = np.random.normal(0, 1, len(t))
    try:
        sos = butter(4, 5000 / (self.sr/2), btype='high', output='sos')
        sig = sosfilt(sos, noise) * np.exp(-decay * t)
    except Exception: pass
elif midi_note in [41, 45, 48]: # Toms
    base_freq = 60.0 if midi_note == 41 else (100.0 if midi_note == 45 else 150.0)
    freqs = (base_freq * 1.5) * np.exp(-10.0 * t) + base_freq
    sig = np.sin(2 * np.pi * np.cumsum(freqs / self.sr)) * np.exp(-3.0 * t)
elif midi_note in [49, 51]: # Cymbals
    decay = 2.5 if midi_note == 49 else 4.0
    noise = np.random.normal(0, 1, len(t))
    try:
        sos = butter(2, [3000/(self.sr/2), 8000/(self.sr/2)], btype='band', output='sos')
        sig = sosfilt(sos, noise) * np.exp(-decay * t)
    except Exception: pass
```

BACKEND MODULE

Instrument Synthesis

```python
def precompute(self, notes: list):
    # 1. Check if a cache file exists for this specific instrument
    cache_file = f"cache_{self.__class__.__name__.lower()}.pkl"

    if os.path.exists(cache_file):
        print(f"   [Cache] Loading {self.__class__.__name__} from disk...")
        with open(cache_file, 'rb') as f:
            self.library = pickle.load(f)
        return

    # 2. If no cache, perform heavy DSP generation
    print(f"   [DSP] Generating {self.__class__.__name__}...")
    for m in notes:
        self.library[m] = self.generate_note(
```

```python
    # 3. Save the result to disk for the nex
    with open(cache_file, 'wb') as f:
        pickle.dump(self.library, f)
```

```python
else:
    actual_midi = round(12 * np.log2(max(freq, 8.0) / 440.0) + 69)
    remainder = actual_midi % 3
    if remainder == 1:
        root_note = actual_midi - 1
        pitch_offset = 1  # Shift up 1 semitone
    elif remainder == 2:
        root_note = actual_midi + 1
        pitch_offset = -1 # Shift down 1 semitone
    else:
        root_note = actual_midi
        pitch_offset = 0

    base_speed = float(2.0 ** (pitch_offset / 12.0))
```

Precomputation strategy

- For non-drum instruments, the nearest precomputed MIDI note is selected and the playback speed is adjusted to match the requested pitch.
- For drums it precomputes a fixed set of MIDI drum hits.

```python
self.instruments = {
    "piano": Piano(sr),
    "guitar": Guitar(sr),
    "strings": Strings(sr),
    "drums": Drums(sr)
}

# NOTE: Using range(24, 105, 3) to compute only every 3rd note!
self.instruments["piano"].precompute(range(24, 105, 3))
self.instruments["guitar"].precompute(range(24, 105, 3))
self.instruments["strings"].precompute(range(24, 105, 3))
self.instruments["drums"].precompute([36, 38, 41, 42, 45, 46, 48, 49, 51])
```

```python
for nid, note in list(self.active_notes.items()):
    data, pos, dlen = note['data'], float(note['pos']), len(note['data'])
    if int(pos) >= dlen - 2:
        dead.append(nid); continue

    total_speed = speed * float(note.get('base_speed', 1.0))

    frac_idx = pos + np.arange(frames, dtype=np.float32) * total_speed
    int_idx = frac_idx.astype(np.int64)
    np.clip(int_idx, 0, dlen - 2, out=int_idx)
    frac = (frac_idx - int_idx).astype(np.float32)
    chunk = data[int_idx] * (1.0 - frac) + data[int_idx + 1] * frac

    if not note['on'] and self.current_vst != "drums":
        rel_t = float(note['rel_pos']) + np.arange(frames, dtype=np.float32) / self.sr
        chunk *= np.exp(-18.0 * rel_t)
        note['rel_pos'] = float(rel_t[-1])
        if note['rel_pos'] > 0.40: dead.append(nid)

    mixed += chunk * float(note.get('gain', 1.0))
```

BACKEND MODULE

Real-time DSP and Effects

FX are applied on the master bus after all active notes are mixed

Tremolo

LFO-controlled volume modulation:
It changes loudness over time

```
t_rel = self.lfo_phase + np.arange(n, dtype=np.float32) / self.sr
lfo = 1.0 - depth * (0.5 - 0.5 * np.cos(2.0 * np.pi * rate * t_rel))
self.lfo_phase = float(t_rel[-1]) % (1.0 / max(rate, 0.01))
```

Delay

Circular buffer echo:
Past audio is read back after a delay time and repeated with feedback.

```
d_samp = max(1, int(delay_time * self.sr))
read_idx = (self.delay_wptr - d_samp + np.arange(n, dtype=np.int64)) % self.delay_maxn
delayed = self.delay_buf[read_idx]
write_idx = (self.delay_wptr + np.arange(n, dtype=np.int64)) % self.delay_maxn

self.delay_buf[write_idx] = sig + feedback * delayed * 0.97
self.delay_wptr = int((self.delay_wptr + n) % self.delay_maxn)

out = sig + level * delayed
```

Reverb

Several comb-filter delay lines simulate dense reflections and spatial tail

```
rev = np.zeros(n, dtype=np.float32)

for i, dlen in enumerate(self.rev_comb_d):
    idx = (self.rev_cptrs[i] + np.arange(n, dtype=np.int64)) % dlen
    comb_out = self.rev_cbufs[i][idx]
    self.rev_cbufs[i][idx] = sig + 0.84 * comb_out
    self.rev_cptrs[i] = int((self.rev_cptrs[i] + n) % dlen)
    rev += comb_out

rev *= 0.125
return ((1.0 - mix) * sig + mix * rev).astype(np.float32)
```

SUMMARY

Evaluation

Strengths

- Clear frontend/backend separation
- WebSocket-to-audio path
- Multiple synthesis methods
- Sequencer, chords, arrangement, recording

Future improvements

- Individual FX and recording
- Better MIDI tempo/track metadata
- More realistic instruments
- Web-based application